

Quectel 云 SDK

版本：1.0

日期：2026-05-20

状态：受控/临时文件

文件保密级别：（请勾选 ☒ ）

绝密 ☐

保密 ☐

公开 ☒

文件管控表

文件变更记录			
修订日期	最新版本	修订内容	编制
2026-05-20	1	首次发行	Wells Li

上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2024，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2024.

目录

1	模块概述	5
2	线程操作	错误!未定义书签。
2.1.	start_new_thread	错误!未定义书签。
2.1.1.	接口概述	5
2.1.2.	函数原型	5
2.1.3.	参数描述	5
2.1.4.	返回值描述	5
2.1.5.	示例	5
2.2.	get_ident	错误!未定义书签。
2.2.1.	接口概述	6
2.2.2.	函数原型	6
2.2.3.	参数描述	6
2.2.4.	返回值描述	6
2.2.5.	示例	6
2.3.	statck_size	错误!未定义书签。
2.3.1.	接口概述	6
2.3.2.	函数原型	6
2.3.3.	参数描述	7
2.3.4.	返回值描述	7
2.3.5.	示例	7
2.4.	join	错误!未定义书签。
2.4.1.	接口概述	7
2.4.2.	函数原型	7
2.4.3.	参数描述	7
2.4.4.	返回值描述	7
2.4.5.	示例	7
3	互斥锁	错误!未定义书签。
3.1.	alloc_lock	错误!未定义书签。
3.1.1.	接口概述	8
3.1.2.	函数原型	8
3.1.3.	参数描述	8
3.1.4.	返回值描述	8
3.2.	lock.acquire	错误!未定义书签。
3.2.1.	接口概述	8
3.2.2.	函数原型	9
3.2.3.	参数描述	9
3.2.4.	返回值描述	9
3.2.5.	示例	9
3.3.	lock.release	错误!未定义书签。
3.3.1.	接口概述	9
3.3.2.	函数原型	9

3.3.3.	参数描述	9
3.3.4.	返回值描述	9
3.3.5.	示例	9
3.4.	lock.locked	错误!未定义书签。
3.4.1.	接口概述	10
3.4.2.	函数原型	10
3.4.3.	参数描述	10
3.4.4.	返回值描述	10
3.4.5.	示例	10
4	信号量	错误!未定义书签。
4.1.	allocate_semaphore	错误!未定义书签。
4.1.1.	接口概述	11
4.1.2.	函数原型	11
4.1.3.	参数描述	11
4.1.4.	返回值描述	12
4.2.	sem.acquire	错误!未定义书签。
4.2.1.	接口概述	错误!未定义书签。
4.2.2.	函数原型	错误!未定义书签。
4.2.3.	参数描述	错误!未定义书签。
4.2.4.	返回值描述	错误!未定义书签。
4.2.5.	示例	错误!未定义书签。
4.3.	sem.release	错误!未定义书签。
4.3.1.	接口概述	错误!未定义书签。
4.3.2.	函数原型	错误!未定义书签。
4.3.3.	参数描述	错误!未定义书签。
4.3.4.	返回值描述	错误!未定义书签。
4.3.5.	示例	错误!未定义书签。
4.4.	sem.getCnt()	错误!未定义书签。
4.4.1.	接口概述	错误!未定义书签。
4.4.2.	函数原型	错误!未定义书签。
4.4.3.	参数描述	错误!未定义书签。
4.4.4.	返回值描述	错误!未定义书签。
4.4.5.	示例	错误!未定义书签。

1 模块概述

云 SDK 模块提供设备接入移远云平台的能力，包括产品信息配置、服务器配置、连接生命周期管理、事件回调注册、物模型数据上报、透传数据上报、OTA 升级相关接口等。用户可通过导入云 SDK 模块完成设备注册、登录、数据上报、事件处理及 OTA 升级流程。

2 初始化与连接管理

2.1. init

2.1.1. 接口概述

初始化云 SDK，读取本地配置文件，加载产品信息、设备密钥、服务器地址、OTA 状态等配置。

2.1.2. 函数原型

```
ret = Qth.init()
```

2.1.3. 参数描述

无。

2.1.4. 返回值描述

初始化成功返回 True,初始化失败异常报错。

2.1.5. 示例

```
import Qth
ret = Qth.init()
if ret:
```

```
print("Qth init success")
```

2.2. start

2.2.1. 接口概述

启动云 SDK 后台线程，执行网络检测、Bootstrap、设备注册、设备登录、重连等流程。调用 `start()` 前，应先完成产品信息配置。

2.2.2. 函数原型

```
ret = Qth.start()
```

2.2.3. 参数描述

无。

2.2.4. 返回值描述

启动成功返回 `True`，启动失败返回 `False`。

2.2.5. 示例

```
import Qth

Qth.init()
Qth.setProductInfo("your_pk", "your_ps")
Qth.setDK("your_dk")

if Qth.start():
    print("Qth start success")
else:
    print("Qth start failed")
```

2.3. stop

2.3.1. 接口概述

停止云 SDK 后台线程，并断开当前 MQTT 连接。

2.3.2. 函数原型

```
Qth.stop()
```


2.3.3. 参数描述

无。

2.3.4. 返回值描述

无。

2.3.5. 示例

```
import Qth
import utime

Qth.init()
Qth.start()

utime.sleep(10)

Qth.stop()
print("Qth stopped")
```

2.4. state

2.4.1. 接口概述

获取当前云 SDK 与平台的连接状态。

2.4.2. 函数原型

```
connected = Qth.state()
```

2.4.3. 参数描述

无。

2.4.4. 返回值描述

已连接云平台返回 **True**；
未连接云平台返回 **False**。

2.4.5. 示例

```
import Qth

if Qth.state():
    print("cloud connected")
```

```
else:
    print("cloud disconnected")
```

3 配置接口

3.1. setProductInfo

3.1.1. 接口概述

设置产品信息，包括 ProductKey 和 ProductSecret。

3.1.2. 函数原型

```
ret = Qth.setProductInfo(pk, ps)
```

3.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
pk	是	str	-	产品 ProductKey
ps	是	str	-	产品 ProductSecret

3.1.4. 返回值描述

设置并保存成功返回 True，保存失败返回 False。

3.1.5. 示例

```
import Qth

Qth.setProductInfo("p123456789", "s123456789")
```

3.2. setDK

3.2.1. 接口概述

设置设备 DK。

3.2.2. 函数原型

```
ret = Qth.setDK(dk)
```

3.2.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
dk	是	str	-	设备 DK

3.2.4. 返回值描述

设置并保存成功返回 True，保存失败返回 False。

3.2.5. 示例

```
import Qth

Qth.setDK("device_001")
```

3.3. setServer

3.3.1. 接口概述

设置云平台服务器地址。

3.3.2. 函数原型

```
ret = Qth.setServer(url)
```

3.3.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
url	是	str	-	MQTT 服务器地址

3.3.4. 返回值描述

设置并保存成功返回 True，保存失败返回 False。

3.3.5. 示例

```
import Qth

Qth.setServer('mqtt://iot-south.quectelcn.com:1883')
```

3.4. setLifetime

3.4.1. 接口概述

设置 MQTT keepalive 生命周期。

3.4.2. 函数原型

```
ret = Qth.setLifetime(lifetime)
```

3.4.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
lifetime	是	int	120	MQTT keepalive 时间，单位秒，范围 21~600

3.4.4. 返回值描述

设置成功返回 True，设置失败返回 False。

3.4.5. 示例

```
import Qth
```

```
Qth.setLifetime(120)
```

3.5. setVer

3.5.1. 接口概述

设置应用版本号，用于 OTA 版本上报和升级结果判断。

3.5.2. 函数原型

```
Qth.setVer(version)
```

3.5.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
version	是	str	-	应用版本号

3.5.4. 返回值描述

无。

3.5.5. 示例

```
import Qth

Qth.setVer("1.0.0")
```

4 事件回调接口

4.1. setEventCb

4.1.1. 接口概述

注册云 SDK 事件回调函数，用于接收设备事件、透传数据、物模型数据、服务调用、OTA 事件等。

4.1.2. 函数原型

```
Qth.setEventCb(eventlist)
```

4.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
eventlist	是	dict	-	回调函数字典

eventlist 支持的字段如下：

字段名	说明
devEvent	设备事件回调
recvTrans	透传数据接收回调
recvTsl	物模型数据下发回调
readTsl	物模型读取回调
readTslServer	物模型服务调用回调
ota.otaPlan	OTA 计划通知回调
ota.fotaResult	OTA 结果回调

4.1.4. 返回值描述

无。

4.1.5. 示例

```
import Qth

def dev_event(event, errcode):
    print("dev event:", event, errcode)

def recv_trans(data):
    print("recv trans:", data)

def recv_tsl(data):
    print("recv tsl:", data)

def read_tsl(id_list, pkgId):
    print("read tsl:", id_list, pkgId)
    qth.ackTsl(1, {1: 25.6}, pkgId)

def read_tsl_server(serverId, value, pkgId):
    print("service:", serverId, value, pkgId)
    qth.ackTslServer(1, serverId, {1: True}, pkgId)

def ota_plan(plans):
    print("ota plans:", plans)

def fota_result(comp_no, result):
    print("ota result:", comp_no, result)

Qth.setEventCb({
    "devEvent": dev_event,
    "recvTrans": recv_trans,
    "recvTsl": recv_tsl,
    "readTsl": read_tsl,
    "readTslServer": read_tsl_server,
    "ota": {
        "otaPlan": ota_plan,
        "fotaResult": fota_result
    }
})
```

5 数据上报接口

5.1. sendTrans

5.1.1. 接口概述

向平台发送透传数据。

5.1.2. 函数原型

```
ret = Qth.sendTrans(mode, value)
```

5.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
mode	是	int	-	QoS 模式, 0 或 1
value	是	bytes / str	-	透传数据内容

5.1.4. 返回值描述

发送成功 返回 True, 发送失败返回 False。

5.1.5. 示例

```
import Qth
```

```
Qth.sendTrans(1, b"hello cloud")
```

5.2. sendTsl

5.2.1. 接口概述

上报物模型数据到平台。

5.2.2. 函数原型

```
ret = qth.sendTsl(mode, value)
```

5.2.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
-----	------	------	-----	----

	填		值	
mode	是	int	-	QoS 模式, 0 或 1
value	是	dict	-	物模型数据字典

5.2.4. 返回值描述

发送成功 返回 True, 发送失败返回 False。

5.2.5. 示例

```
import Qth

# 例如物模型 ID 1 表示温度, ID 2 表示湿度
Qth.sendTsl(1, {
    1: 25.6,
    2: 60.5
})
```

5.3. ackTsl

5.3.1. 接口概述

回复平台物模型读取请求, 通常在 readTsl 回调中调用。

5.3.2. 函数原型

```
ret = Qth.ackTsl(mode, value, pkgId)
```

5.3.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
mode	是	int	-	QoS 模式, 0 或 1
value	是	dict	-	回复的物模型数据
pkgId	是	int	-	平台请求包 ID, 范围 1~65535

5.3.4. 返回值描述

回复成功 返回 True, 回复失败返回 False。

5.3.5. 示例

```
import Qth
```



```
def read_tsl(id_list, pkgId):
    data = {}

    if 1 in id_list:
        data[1] = 25.6

    if 2 in id_list:
        data[2] = 60.5

    Qth.ackTsl(1, data, pkgId)
```

5.4. ackTslServer

5.4.1. 接口概述

回复平台物模型服务调用请求，通常在 readTslServer 回调中调用。

5.4.2. 函数原型

```
ret = Qth.ackTslServer(mode, serverId, value, pkgId)
```

5.4.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
mode	是	int	-	QoS 模式, 0 或 1
serverId	是	int	-	服务 ID
value	是	dict	-	回复的物模型数据
pkgId	是	int	-	平台请求包 ID, 范围 1~65535

5.4.4. 返回值描述

回复成功 返回 True，回复失败返回 False。

5.4.5. 示例

```
import Qth

def read_tsl_server(serverId, value, pkgId):
    print("service called:", serverId, value)

    Qth.ackTslServer(1, serverId, value, pkgId)
```

5.5. sendOutsideLocation

5.5.1. 接口概述

上报外部定位信息。

5.5.2. 函数原型

```
ret = Qth.sendOutsideLocation(location)
```

5.5.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
location	是	str	-	外部定位数据

5.5.4. 返回值描述

上报成功 返回 True，上报失败返回 False。

5.5.5. 示例

```
import Qth
gpgga = "$GPGGA,031025.000,3115.6422,N,12127.5494,,1,6,1.93,128.6,,8.1,M,,*6F"
Qth.sendOutsideLocation(gpgga)
```

6 OTA 接口

6.1. otaRequest

6.1.1. 接口概述

主动向平台请求 OTA 升级计划。

6.1.2. 函数原型

```
ret = Qth.otaRequest()
```

6.1.3. 参数描述

无。

6.1.4. 返回值描述

请求成功 返回 True，请求失败返回 False。

6.1.5. 示例

```
import Qth

Qth.otaRequest()
```

6.2. otaAction

6.2.1. 接口概述

上报物模型数据到平台。

6.2.2. 函数原型

```
ret = Qth.otaAction(action)
```

6.2.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
action	是	int	-	OTA 操作，1 表示确认升级，0 表示拒绝升级

6.2.4. 返回值描述

操作成功 返回 True，操作失败返回 False。

6.2.5. 示例

```
import Qth

# 确认升级
Qth.otaAction(1)

# 拒绝升级
# Qth.otaAction(0))
```

6.3. apply_pending_update

6.3.1. 接口概述

应用待升级的 APP OTA 文件。该接口通常建议在 `boot.py` 启动早期调用，用于检测是否存在 OTA 升级标志文件，并将 `staging` 目录中的文件覆盖到目标目录。

6.3.2. 函数原型

```
ret = Qth.apply_pending_update()
```

6.3.3. 参数描述

无。

6.3.4. 返回值描述

存在待升级文件，且应用成功返回 `True`，
无待升级文件，或应用失败 返回 `False`。

6.3.5. 示例

```
import qth

qth.apply_pending_update()
```